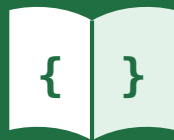

Capstone — Bookstore Platform

Guide

Spring Boot Edu



2026-09-25

Contents

1. Learning Goals	2
2. Concepts	2
2.1 The Platform at a Glance	2
2.2 Where the Course Modules Meet	2
3. Step-by-Step Examples	3
3.1 Start the Platform	3
3.2 Contracts: gRPC and Events	4
3.3 Placing an Order: gRPC Reservation and Outbox	5
3.4 Stock Under a Distributed Lock	6
3.5 Events: the Outbox Relay and BookChanged	7
3.6 The Search Read Model and its Cache	8
3.7 The Gateway: Routes, JWT and Rate Limit	9
3.8 One Trace Through Everything	10
3.9 Testing a Distributed System	11
3.10 On Kubernetes with Helm	12
4. Common Mistakes and Best Practices	13
5. Summary	13
6. Further Reading	13

1. Learning Goals

By the end of the capstone you will be able to:

- run a system of four Spring Boot services and six infrastructure systems with one command, and on Kubernetes with Helm
- follow one order through REST, gRPC, a transactional outbox, Kafka, Elasticsearch and a Redis cache
- explain why each technology sits where it does (the architecture document has the decisions: ADR-1 ... ADR-8)
- test a distributed system on three levels: one service with fakes, one service with real infrastructure, the whole platform end to end
- find one request in Grafana as a single trace across all services

Prerequisites: modules 00–24 (the capstone uses them all) · **Estimated time:** 8–12 hours with the exercises

2. Concepts

2.1 The Platform at a Glance

The architecture document (`docs/en/architecture.md`) describes the system and its decisions; this guide walks through the code. In short:

Service	Owns	Talks to
gateway	the only way in	all services (HTTP), Redis (rate limit)
order-service	orders (PostgreSQL)	catalog (gRPC), Kafka (outbox relay, notifications)
catalog-service	books and stock (MongoDB)	Hazelcast (locks), Kafka (BookChanged)
search-service	the search read model (Elasticsearch)	Kafka (consumer), Redis (cache)

Every service has its own database. The gateway checks the JWT first; the order and catalog services check it again (the search API is public).

2.2 Where the Course Modules Meet

Topic	Module	In the capstone
JPA, Flyway, transactions	05, 06	orders, outbox, notifications
MongoDB	07	catalog, reservations
Redis	08	search cache, rate limiter
Hazelcast IMap lock	09	stock per ISBN
Elasticsearch	10	search read model
Kafka, outbox, idempotent consumers	11	OrderPlaced, BookChanged
JWT resource server	12	gateway, order and catalog services
Testcontainers, fakes	14	tests of every service, the end-to-end test
OpenTelemetry, LGTM	15	one trace per order
Docker, jlink	20	one Dockerfile for all services
gRPC	21	stock reservation
Kubernetes, probes	22	the Helm chart
Spring Cloud Gateway	23	the gateway

i Note

The capstone does not introduce new libraries. Everything here was explained in a module; the new part is how the pieces work together — and what goes wrong between them.

3. Step-by-Step Examples

3.1 Start the Platform

One command builds the four images from source (Maven runs inside Docker) and starts everything:

```
cd capstone && docker compose up --build          # first build: ~10 minutes, afterwards  
↪ seconds
```

During development, building the jars on the host is faster (capstone/up.sh does it and starts compose with JAR_SOURCE=host). The gateway listens on port 8080, Grafana on port 3000; nothing else is published. Try the flow with capstone/requests.http, or with curl:

```
TOKEN=$(curl -s -X POST localhost:8080/api/auth/token -H 'Content-Type:
↪ application/json' \
    -d '{"username":"ayse","password":"ayse-demo"}' | jq -r .accessToken)
curl -s "localhost:8080/api/search?q=kafka" # "sold": 0
curl -s -X POST localhost:8080/api/orders -H "Authorization: Bearer $TOKEN" \
    -H 'Content-Type: application/json' -d
    ↪ '{"lines":[{"isbn":"9781492078005","quantity":2}]}'
curl -s "localhost:8080/api/search?q=kafka" # a moment later:
↪ "sold": 2
```

★ Tip

The token endpoint exists only in the dev profile of the gateway (ADR-5). The demo passwords come from `application-dev.yaml` and can be overridden with environment variables.

3.2 Contracts: gRPC and Events

Services share only contracts, never classes of each other. The contracts library holds the gRPC service of the catalog ...

```
service StockService {
    // Reserves all lines or none. Idempotent per order_ref: a retry returns the first
    ↪ result.
    // NOT_FOUND: unknown ISBN · FAILED_PRECONDITION: not enough stock
    rpc ReserveStock (ReserveStockRequest) returns (ReserveStockResponse);
    // Gives the stock of a reservation back (when the order could not be saved).
    rpc ReleaseStock (ReleaseStockRequest) returns (ReleaseStockResponse);
}
```

... and the events as plain records. They travel as JSON without type headers, so any consumer — in any language — can read them:

```
public record OrderPlaced(String orderId, String customerId, List<Line> lines,
↪ BigDecimal total, Instant placedAt) {

    public static final String TOPIC = "bookstore.orders";

    public record Line(String isbn, String title, int quantity, BigDecimal unitPrice) {
    }
}
```

! Important

A contract changes more slowly than the code around it. Add fields, never rename or remove them; a service that is deployed later must still understand the old messages.

3.3 Placing an Order: gRPC Reservation and Outbox

The order service reserves the stock first and saves second. The reservation uses the order's ID as its key, so a repeated call reserves nothing new:

```
OrderResponse place(String customerId, PlaceOrderRequest request) {
    UUID id = UUID.randomUUID(); // also the idempotency key of
    ↪ the reservation
    Map<String, Integer> quantities = request.lines().stream().collect(Collectors.toMap(
        PlaceOrderRequest.Line::isbn, PlaceOrderRequest.Line::quantity, Integer::sum,
    ↪ LinkedHashMap::new));

    // the remote call runs OUTSIDE the transaction: no database connection is held while
    ↪ we wait
    List<ReservedBook> reserved = reserveOrGiveBack(id, quantities);
    try {
        return transactions.execute(status -> save(id, customerId, reserved));
    } catch (RuntimeException e) {
        releaseAfterFailure(id, e); // compensation: the catalog
        ↪ gets its copies back
        throw e;
    }
}
```

Three details matter:

- The gRPC call happens **outside** the transaction. A database connection is never held while another service works.
- The order and its OrderPlaced event are saved in **one** transaction (ADR-3). There is no moment in which one exists without the other.
- If saving fails, the reservation is released — a compensation, because there is no distributed transaction.

The client of the stock service sets a deadline on every call and forwards the customer's JWT (the catalog checks it too):

```
StockClient(stockServiceGrpc.StockServiceBlockingStub stock, CatalogProperties catalog)
↪ {
    // the interceptor asks for the token on every call: it is the token of the current
    ↪ request
```

```

    this.stock = stock.withInterceptors(new BearerTokenAuthenticationInterceptor(
        (Supplier<String>) StockClient::currentToken));
    this.deadline = catalog.deadline();
}

```

gRPC status codes become HTTP answers in OrderErrors: FAILED_PRECONDITION → 409, NOT_FOUND → 422, UNAVAILABLE/DEADLINE_EXCEEDED → 503.

3.4 Stock Under a Distributed Lock

Two catalog instances run side by side (two replicas on Kubernetes). Without a lock, both could sell the last copy. The catalog locks every ISBN of the order in Hazelcast, always in the same (sorted) order, so that two orders cannot wait for each other forever:

```

public Reservation reserve(String orderRef, Map<String, Integer> quantities) {
    var existing = reservations.findById(orderRef);
    if (existing.isPresent()) {
        return existing.get(); // idempotent: a retry
        ↪ of the same order
    }
    Map<String, Integer> sorted = new TreeMap<>(quantities); // always the same
    ↪ lock order
    List<String> locked = new ArrayList<>();
    try {
        sorted.keySet().forEach(isbn -> locked.add(lock(isbn)));
        List<Book> selected = sorted.entrySet().stream().map(entry ->
            ↪ available(entry.getKey(), entry.getValue())).toList();
        List<Reservation.Line> lines = new ArrayList<>();
        for (Book book : selected) {
            int quantity = sorted.get(book.isbn());
            books.save(book.withStock(book.stock() - quantity));
            lines.add(new Reservation.Line(book.isbn(), book.title(), quantity,
            ↪ book.price()));
        }
        return reservations.save(new Reservation(orderRef, lines,
            ↪ Reservation.Status.RESERVED));
    } finally {
        locked.forEach(locks::unlock);
    }
}

```

StockServiceTest proves it: ten parallel orders for a book with five copies — exactly five succeed.

! Warning

The lock has a lease time (10 s). A crashed instance releases its locks automatically — but a reservation that takes longer than the lease is no longer protected. Keep the locked section short: no remote calls inside it.

3.5 Events: the Outbox Relay and BookChanged

A scheduled relay publishes the outbox rows and marks them as sent. FOR UPDATE SKIP LOCKED lets several order-service instances relay in parallel without sending a row twice:

```
@Scheduled(fixedDelayString = "${bookstore.outbox.relay-interval}")
@Transactional
public void publishPending() throws ExecutionException, InterruptedException,
    ↪ TimeoutException {
    List<OutboxEvent> pending = events.lockNextToPublish();           // other
    ↪ instances skip these rows
    for (OutboxEvent event : pending) {
        kafka.send(event.getTopic(), event.getEventKey(), event.getPayload())
            .get(10, TimeUnit.SECONDS);                             // wait for the
        ↪ broker's acknowledgement
        event.markPublished(clock.instant());                         // saved at commit
    ↪ (dirty checking)
    }
}
}
```

The catalog publishes BookChanged after **every** save of a book, with a Spring Data MongoDB lifecycle event. Admin changes, the seed data and the stock reservations all go through the repository, so none of them can be forgotten:

```
@Override
public void onAfterSave(AfterSaveEvent<Book> event) {
    Book book = event.getSource();
    var changed = new BookChanged(book.isbn(), book.title(), book.authors(),
    ↪ book.description(), book.price(),
        book.stock());
    kafka.send(BookChanged.TOPIC, book.isbn(), changed)
        .whenComplete((result, failure) -> {
            if (failure != null) {
                log.warn("BookChanged for {} not published: {}", book.isbn(),
    ↪ failure.getMessage());
            }
        });
}
```

i Note

The catalog has no outbox — a deliberate trade-off. If Kafka is down, the search index misses the change until the book changes again. For orders that would be unacceptable; for a search index it is tolerable. Compare the two approaches.

3.6 The Search Read Model and its Cache

The search service owns no data: its index is built only from events (ADR-4). Counting sales must survive redelivered messages — Kafka delivers at least once — so Redis remembers which orders were counted (SET NX), and Elasticsearch adds the numbers atomically with a script:

```
@CacheEvict(cacheNames = "search", allEntries = true)
public void recordSale(OrderPlaced order) {
    String mark = "search:sale-counted:" + order.orderId();
    if (!Boolean.TRUE.equals(redis.opsForValue().setIfAbsent(mark, "1",
        ↪ REMEMBER_SALES))) {
        return; // counted before
    }
    try {
        order.lines().forEach(line ->
        ↪ elasticsearch.update(UpdateQuery.builder(line.isbn())
            .withScript("ctx._source.sold += params.quantity") // atomic inside
            ↪ Elasticsearch
            .withParams(Map.of("quantity", line.quantity()))
            .withRefreshPolicy(RefreshPolicy.IMMEDIATE)
            .build(), BOOKS));
    } catch (RuntimeException e) {
        // let the redelivery try again. Caveat: lines updated before the failure are
        ↪ counted twice then -
        // exact counting would need one document per order (or per order line)
        ↪ instead of a counter
        redis.delete(mark);
        throw e;
    }
}
```

Search results are cached in Redis per normalized query. Every index change evicts the cache (@CacheEvict(allEntries = true) on the index methods), so a new sale is visible at once:

```
@Cacheable(cacheNames = "search", key = "#query.strip().toLowerCase()") // "Java"
    ↪ and " java" share an entry
public SearchResult search(String query) {
```

```

NativeQuery search = NativeQuery.builder()
    .withQuery(q -> q.multiMatch(match -> match
        .query(query)
        .fields("title^3", "authors^2", "description")    // a title
        ↪ match counts most
        .fuzziness("AUTO")))                            // "effective"
        ↪ still finds "effective"
    .withMaxResults(20)
    .build();
var hits = elasticsearch.search(search, BookDocument.class).stream()
    .map(SearchHit::getContent).map(BookHit::from).toList();
return new SearchResult(query, hits);
}
}

```

3.7 The Gateway: Routes, JWT and Rate Limit

The gateway routes by path. The same rate limiter protects every route:

```

routes:
- id: orders
  uri: ${bookstore.services.orders}
  predicates:
  - Path=/api/orders/**
  filters:
  - &rate-limit
    name: RequestRateLimiter           # token bucket in Redis, shared by all
    ↪ gateway instances
    args:
      redis-rate-limiter.replenishRate: ${bookstore.rate-limit.replenish-rate}
      redis-rate-limiter.burstCapacity: ${bookstore.rate-limit.burst-capacity}
      key-resolver: "#{@customerOrIpKeyResolver}"
- id: catalog
  uri: ${bookstore.services.catalog}
  predicates:
  - Path=/api/catalog/**
  filters:
  - *rate-limit
- id: search
  uri: ${bookstore.services.search}
  predicates:
  - Path=/api/search/**
  filters:
  - *rate-limit

```

Whose bucket does a request use? A signed-in customer's own, so one customer cannot

slow down the others; an anonymous request the bucket of its IP address:

```
@Bean
KeyResolver customerOrIpKeyResolver() {
    return exchange -> exchange.getPrincipal()
        .map(Principal::getName)
        .map(subject -> "customer:" + subject)
        .switchIfEmpty(Mono.fromSupplier(() -> "ip:" +
            ↪ Optional.ofNullable(exchange.getRequest().getRemoteAddress())
                .map(InetSocketAddress::getHostString).orElse("unknown"))));
}
```

The gateway's security chain is the first check (401 without a token, 403 for a customer who changes the catalog). The Authorization header is forwarded unchanged, and the order and catalog services check it again — defence in depth (ADR-5).

3.8 One Trace Through Everything

Every service exports metrics and traces over OTLP to Grafana LGTM. Kafka observation carries the trace context in the record headers:

```
tracing:
  sampling:
    probability: 1.0 # every request is traced (production: e.g.
    ↪ 0.1)
  observations:
    enable:
      tasks.scheduled: false # no trace for every relay run (twice a
      ↪ second)
  metrics:
    tags:
      application: ${spring.application.name}
  opentelemetry:
    tracing:
      export:
        otlp:
          endpoint: ${OTLP_ENDPOINT:http://localhost:4318}/v1/traces
  otlp:
    metrics:
      export:
        url: ${OTLP_ENDPOINT:http://localhost:4318}/v1/metrics
        step: 10s
```

Place an order, open Grafana (<http://localhost:3000>) → Explore → Tempo, and search for the service gateway. The trace of `POST /api/orders` shows the gateway, the order ser-

vice, the gRPC call `StockService/ReserveStock` in the catalog, the Kafka message `bookstore.catalog_send` and the search service updating its index — five services, one trace.

i Note

The `OrderPlaced` event starts a new trace: the outbox row does not carry the trace context, and the relay runs on its own schedule. Carrying it through is an extra challenge in the exercises.

3.9 Testing a Distributed System

The capstone is tested on three levels:

1. **One service with fakes.** The order service's tests start a fake `StockService` in-process (Spring gRPC's test transport). The fake records what it was asked, so a test can check the forwarded token or the released reservation:

```
@TestConfiguration(proxyBeanMethods = false)
class FakeCatalog {

    public static final FakeStockService STOCK = new FakeStockService();

    @Bean
    @GrpcService
    FakeStockService fakeStockService() {
        return STOCK;
    }

    @Bean
    @GlobalServerInterceptor
    FakeStockService.Recorder recordHeaders() {
        return new FakeStockService.Recorder(STOCK);
    }
}
```

2. **One service with real infrastructure.** Testcontainers starts PostgreSQL, MongoDB, Elasticsearch, Redis, Kafka or Hazelcast — whatever the service really uses. A small `TopicReader` reads the events as raw JSON, the way another service would see them.
3. **The whole platform.** PlatformIT builds the four images from the jars of the current build, starts `compose.yaml` (without LGTM) and drives one order through the gateway:

```
static final ComposeContainer PLATFORM = new ComposeContainer(
    new File("../compose.yaml"), new File("compose.e2e.yaml"))
    .withBuild(true) // images from the jars of
    ↪ this build
```

```

.withServices("gateway") // and everything it depends
↪ on (not LGTM)
.withExposedService("gateway", 8080, Wait.forHttp("/actuator/health/readiness")
    .forStatusCode(200).withStartupTimeout(Duration.ofMinutes(10)));

```

★ Tip

Most bugs live between services, but most tests should not. Keep the end-to-end test to the main flow; test the details in the fast service tests.

3.10 On Kubernetes with Helm

The Helm chart `k8s/helm/bookstore` has one generic template for the four services and one for the development infrastructure. Each service is a small block of values:

```

catalog-service:
  image: springbootedu/capstone-catalog-service:latest
  replicas: 2 # two instances share the stock locks in
↪ Hazelcast (ADR-6)
  grpcPort: 9090
  env:
    MONGODB_URI: mongodb://mongo:27017/catalog
    HAZELCAST_ADDRESS: hazelcast:5701
  waitFor: [mongo:27017, hazelcast:5701, kafka:9092]

```

`capstone/k8s/deploy-kind.sh` builds the images, loads them into `kind` and runs `helm upgrade --install`. Two catalog replicas share the Hazelcast locks; the order service reaches both over a headless Service, and gRPC balances its calls with round robin:

```

@Bean
<T extends ManagedChannelBuilder<T>> GrpcChannelBuilderCustomizer<T>
↪ catalogRoundRobin() {
  return GrpcChannelBuilderCustomizer.matching("catalog", builder ->
    ↪ builder.defaultLoadBalancingPolicy("round_robin"));
}
}

```

!! Caution

Kubernetes injects variables such as `KAFKA_PORT=tcp://10.96....` for every Service. The Kafka image reads every `KAFKA_*` variable as configuration, and `${REDIS_PORT:6379}` suddenly resolves to a URL. The chart sets `enableServiceLinks: false` on every pod.

4. Common Mistakes and Best Practices

- `□` Calling another service inside a database transaction. `□` Remote call first (or after the commit), then a short transaction.
- `□` Publishing an event directly after saving (`save()` then `kafkaTemplate.send()`): one of the two can fail. `□` An outbox row in the same transaction, published by a relay.
- `□` Consumers that assume exactly-once delivery. `□` Idempotent consumers: a primary key, `ON CONFLICT DO NOTHING`, a `SET NX` mark.
- `□` Two replicas that seed the same data with “count, then save all”: the second crashes on a duplicate key. `□` Insert per document and ignore `DuplicateKeyException` (`CatalogSeeder`).
- `□` A gRPC client with the default pick-first policy behind a headless Service: every call goes to the same pod. `□` `round_robin`.
- `□` Secrets with defaults in `application.yaml`. `□` `${BOOKSTORE_JWT_SECRET}` without a default; `compose` sets a development value, the Helm chart generates one.
- `□` An end-to-end test for every detail. `□` One flow end to end, the details in service tests.

5. Summary

- The capstone combines the course: four services, six infrastructure systems, one command (`docker compose up --build`), and the same images on Kubernetes with Helm.
- An order is reserved synchronously over gRPC and published asynchronously through an outbox; the search index is a read model built from events.
- Distributed systems need idempotency everywhere: reservations by order ID, sales by order ID, notifications by primary key.
- Observability connects the pieces: one trace from the gateway to the search index.
- Tests on three levels keep the system changeable: fakes, `Testcontainers`, one end-to-end flow.

6. Further Reading

- The architecture and decisions of this platform: `docs/en/architecture.md`
- Transactional outbox pattern: <https://microservices.io/patterns/data/transactional-outbox.html>
- Spring gRPC reference: <https://docs.spring.io/spring-grpc/reference/>
- Spring Cloud Gateway — `RequestRateLimiter`: <https://docs.spring.io/spring-cloud-gateway/reference/spring-cloud-gateway-server-webflux/gatewayfilter-factories/requestratelimiter-factory.html>
- Hazelcast — locking maps: <https://docs.hazelcast.com/hazelcast/5.5/data-structures/locking-maps>

- Helm documentation: <https://helm.sh/docs/>